

Payer Alias Resolution

Users will type “BCBS”, “United”, or “Aetna Better Health” — not the exact `PayerDisplayName` stored in the reimbursement tables. This is where that translation should live, and why it does not belong in the agent's prompt or in the dashboard.

Proposal · Phase 7 · pre-launch · Reuses `dbo.PayerFamilyRule` · No UI change

RECOMMENDATION

Build it into the bridge as a database resolver the agent calls as a tool — not into the Foundry instructions, and not into the dashboard.

The rules already exist in `dbo.PayerFamilyRule`, where business users maintain them without a deployment. The bridge is the one place that can read them live, hand the agent an exact answer, and serve every client at once. The agent side changes by a single instruction; the dashboard does not change at all.

The problem

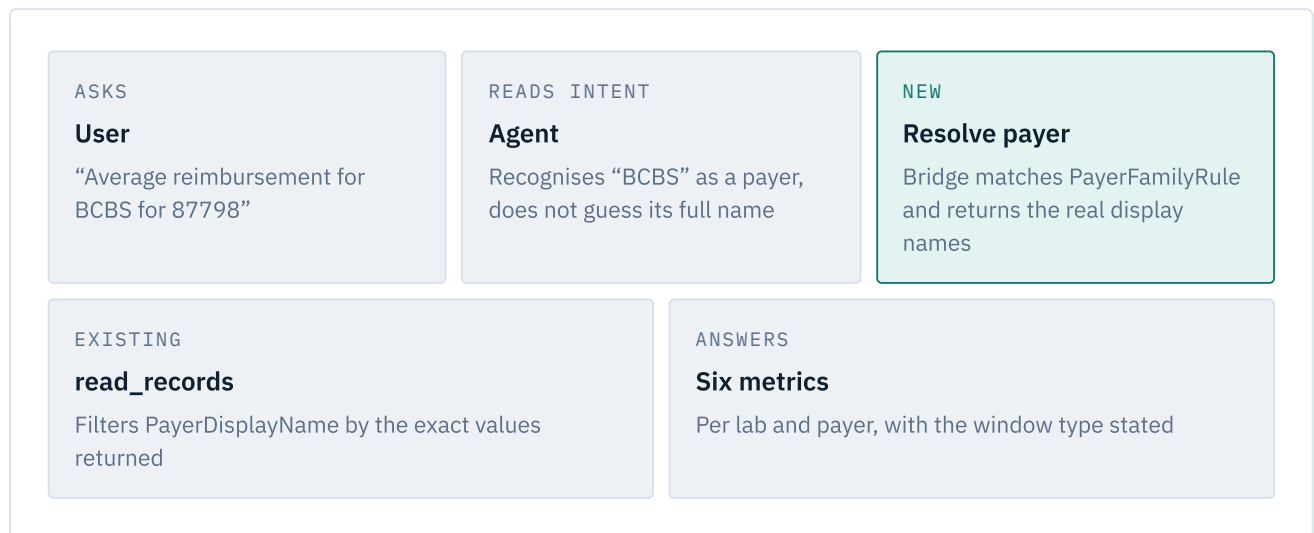
Every reimbursement entity — `CPTAverage`, `CPTPricingStatistics`, `PanelPricingStatistics`, `ComparePayerPricingForCPT` — filters on `PayerDisplayName`, which holds the payer's full stored name. A user asking “*average reimbursement for BCBS for 87798*” supplies a short form that matches no row, so the agent returns “no matching data found” for a payer that is plainly in the data.

The knowledge needed to fix this is already written down. `dbo.PayerFamilyRule` holds a `Pattern` column of pipe-separated aliases against a `Family`, ordered by `Priority` so a sub-brand beats its parent — `AETNA_BETTER_HEALTH` at priority 10 resolves before `AETNA` at 50. The Payer Policy Mapper already uses exactly this table for the same purpose on the claims side.

Where it could go

OPTION	WHAT IT WOULD MEAN	WHY / WHY NOT	VERDICT
Foundry agent system instructions	Paste the alias list into the agent's prompt and let the model match names itself.	A prompt is static text — it would be a snapshot of a live table, stale the first time someone adds a brand, and it needs an agent edit and a retest every time. It also asks the model to decide which payer was meant, which is the failure we have already fixed four times (invented field names, self-computed statistics).	NO
Dashboard LabMetricsDashboard	Rewrite the user's wording before sending the question to the agent.	The payer name is buried inside free-form English, so the dashboard would first have to work out which words are the payer — a language-parsing job it has no business doing. It would also only work for this one screen: the Foundry Playground and any future caller would still be broken.	NO
The bridge Data API Builder → MCP	A stored procedure reads PayerFamilyRule and returns the real PayerDisplayName values. Exposed through the bridge, it becomes a tool the agent can call, alongside read_records .	The rules and the data are both already here, so there is nothing to duplicate and nothing to keep in sync. Business users keep editing the rules table with no deployment. The answer is exact rather than inferred, and every client — the chat screen, the Playground, anything later — gets it for free.	RECOMMENDED

How it works



Only the third box is new. The agent still asks the same questions of the same data; it simply stops guessing at the payer name and asks the database instead.

The output contract

The resolver returns `PayerDisplayName` values, deliberately — it is the one column present on all four entities, so a single resolver serves CPT, panel and payer-comparison questions alike. When an alias matches several payers, it returns all of them, and the agent's existing per-payer breakdown rule takes over from there. When nothing matches, it returns nothing, and the agent says so plainly rather than inventing a figure.

Confirm this first

ONE OPEN QUESTION

There are two ways to get from a matched family to the real payer names, and a single query decides which. If `CPTAverage.Global_Payer_ID` is reliably populated, we can join through `PayerPolicyInsuranceMaster` and inherit its `PayerFamily` directly — the cleaner path, and consistent with the claims side. If coverage is patchy, we match the alias tokens against `PayerDisplayName` instead, which needs nothing extra but is a text match.

```
-- Run against LRNMaster. If NullIds is near zero, take the join path.  
SELECT COUNT(*) AS Rows,  
       SUM(CASE WHEN Global_Payer_ID IS NULL THEN 1 ELSE 0 END) AS NullIds,  
       COUNT(DISTINCT PayerDisplayName) AS DistinctPayers  
FROM dbo.CPTAverage;
```

I have not run this — it needs database access. It is the first thing to check, because it settles the shape of the stored procedure below.

Implementation

Four steps, in order — each one depends on the one before it.

Add the resolver stored procedure

`dbo.usp_AI_ResolvePayerName` takes the user's words and returns the payer names that actually exist in the data. It canonicalises the input the same way the Payer Policy Mapper does — uppercase, punctuation to spaces, collapse runs — then walks `PayerFamilyRule` in `Priority` order and takes the first hit, so sub-brands keep beating their parents.

```
-- Sketch, not final: written from the schema, not yet run against LRNMaster.
CREATE OR ALTER PROCEDURE dbo.usp_AI_ResolvePayerName
    @PayerText NVARCHAR(200)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Canonical NVARCHAR(200) = UPPER(LTRIM(RTRIM(@PayerText)));

    -- Pattern is a pipe-separated alias list, so split it into one row per alias.
    WITH Alias AS (
        SELECT r.Family, r.Priority, r.RuleId,
               LTRIM(RTRIM(s.value)) AS Token
        FROM dbo.PayerFamilyRule r
        CROSS APPLY STRING_SPLIT(r.Pattern, '/') s
        WHERE r.IsActive = 1 AND LTRIM(RTRIM(s.value)) <> ''
    ),
    Hit AS (
        -- lowest Priority wins: sub-brand over parent
        SELECT TOP (1) Family
        FROM Alias
        WHERE @Canonical LIKE '%' + Token + '%'
        ORDER BY Priority, RuleId
    )
    SELECT DISTINCT h.Family, c.PayerDisplayName
    FROM Hit h
    JOIN Alias a ON a.Family = h.Family
    JOIN dbo.CPTAverage c
        ON UPPER(c.PayerDisplayName) LIKE '%' + a.Token + '%';
END
```

Union in `PanelPricingStatistics` so panel questions resolve too. If the coverage check above comes back clean, replace the final `LIKE` join with a join on `Global_Payer_ID`.

Expose it through the bridge

Data API Builder can publish a stored procedure as an entity, which is what turns it into a tool the agent can call. Add it to `dab-config.json` alongside the four existing entities, grant `dab_reader` execute rights, then rebuild and push the image as `:v3` — the same path already used to go from `:latest` to `:v2`.

```
-- Grant, then rebuild the image and update the container app.  
GRANT EXECUTE ON dbo.usp_AI_ResolvePayerName TO dab_reader;
```

Nothing about API Management, the subscription key, or the agent's MCP connection changes — the new tool arrives on the address the agent already uses.

Add one rule to the agent

This is the entire agent-side change. It follows the pattern that has already worked for us: forbid the guess, name the tool that gives the real answer.

Resolving payer names

When a question names a payer, never filter `PayerDisplayName` with the user's own words. Call `resolve_payer_name` first and use only the exact `PayerDisplayName` values it returns.

If it returns several payers, report them all, broken out per payer as usual. If it returns none, say that no payer matching that name was found — do not fall back to a guessed or partial name.

Test the cases that matter

Confirm in the Foundry Playground before touching the chat screen, since the Playground isolates the agent from everything else in the chain.

Must pass

- A short form — “BCBS for 87798” — returns the same figures as the full name.
- A sub-brand — “Aetna Better Health” — resolves to itself, not to Aetna.
- An exact full name still works, unchanged.
- An unknown payer says so, rather than answering about a different one.

Watch for

- An alias matching several payers — the answer must break out per payer, not merge them.
- A panel question with an alias, to confirm the panel entity resolves too.
- Response time, since this adds one tool call per question.

Scope and effort

What changes

- One new stored procedure in LRNMaster.
- One new entity in the bridge's config, and a rebuilt container image.
- One new section in the agent's instructions.

What does not

- The dashboard — no code, no redeploy.
- The backend proxy on App Service.
- API Management, its key, or the agent's MCP connection.
- `PayerFamilyRule` itself — read only, and it stays editable without a deploy.

Roughly one to two days of developer time end to end, most of it in the stored procedure and its testing. The riskiest step is the bridge rebuild, and that is a path we have already run once.

Known risks

Pattern syntax

The patterns are plain alias lists today, which is why splitting on the pipe works. A rule using genuine regular-expression syntax would not translate to SQL, so the resolver should be reviewed against the live table before release.

One extra call

Each payer question costs one additional round trip to the bridge. It is a small indexed lookup against a short rules table, but it is worth measuring rather than assuming.

Prepared for review. The open coverage question in “Confirm this first” should be answered before work starts — it determines whether the resolver joins on `Global_Payer_ID` or matches on name, and that is the only structural decision left.